

Практическая работа: автотесты Python-модуля

Результат: вы создаёте, запускаете и интерпретируете набор pytest-тестов для модуля заказов.

ОРИЕНТИРЫ

На чём сосредоточиться

- Факт вместо общего впечатления.
- Проверяемый ожидаемый результат.
- Связь проверки с риском.

ПОНЯТИЯ

Рабочий словарь

Arrange–Act–Assert

Структура: подготовить данные, выполнить действие, проверить результат.

Изоляция

Независимость теста от внешнего состояния и других тестов.

Регрессионный тест

Тест, сохраняющий найденное правило после исправления дефекта.

Падение теста

Факт несоответствия ожидания и результата, требующий исследования.

КОНТЕКСТ

Учебный кейс

В сервисе появилась жалоба: пользователь отменил уже отправленный заказ. Защитите правило модульными тестами, затем объясните, какой тест должен оставаться после исправления.

ПРИМЕР

Данные для проверки

```
# orders.py
def can_cancel(status: str) -> bool:
    return status in {"new", "paid"}

# Ошибка в исходной версии: статус shipped тоже возвращал True.
```

РАЗБОР

Главная мысль

Работайте по инструкции, фиксируя входные данные, действие и наблюдаемый результат.

Сравнивайте готовый артефакт с критериями самопроверки.

ДЕЙСТВИЕ

Практический маршрут

- Создайте `test_orders.py` и добавьте импорт `can_cancel`.
- Подготовьте параметризованный набор допустимых и недопустимых статусов.
- Запустите `pytest -q` и сохраните вывод до исправления.
- Исправьте правило только после того, как тест воспроизвёл проблему.
- Запустите набор повторно и добавьте тест для неизвестного статуса.

САМОПРОВЕРКА

Перед тем как сделать вывод

- Указаны ли конкретные входные данные?
- Можно ли повторить проверку?
- Связано ли решение с риском качества?